Polymove - Microservices Lab 4: Message-Driven Architecture

Introduction

In the previous labs, all communication between services was **synchronous**: REST calls wait for a response, gRPC calls block until completion. This creates tight coupling — if one service is slow or down, the caller is affected.

In this lab, you will implement **asynchronous, event-driven communication** using a message broker. Services will publish events without knowing who consumes them, and multiple services can react to the same event independently.

Learning Objectives

By the end of this lab, you will have:

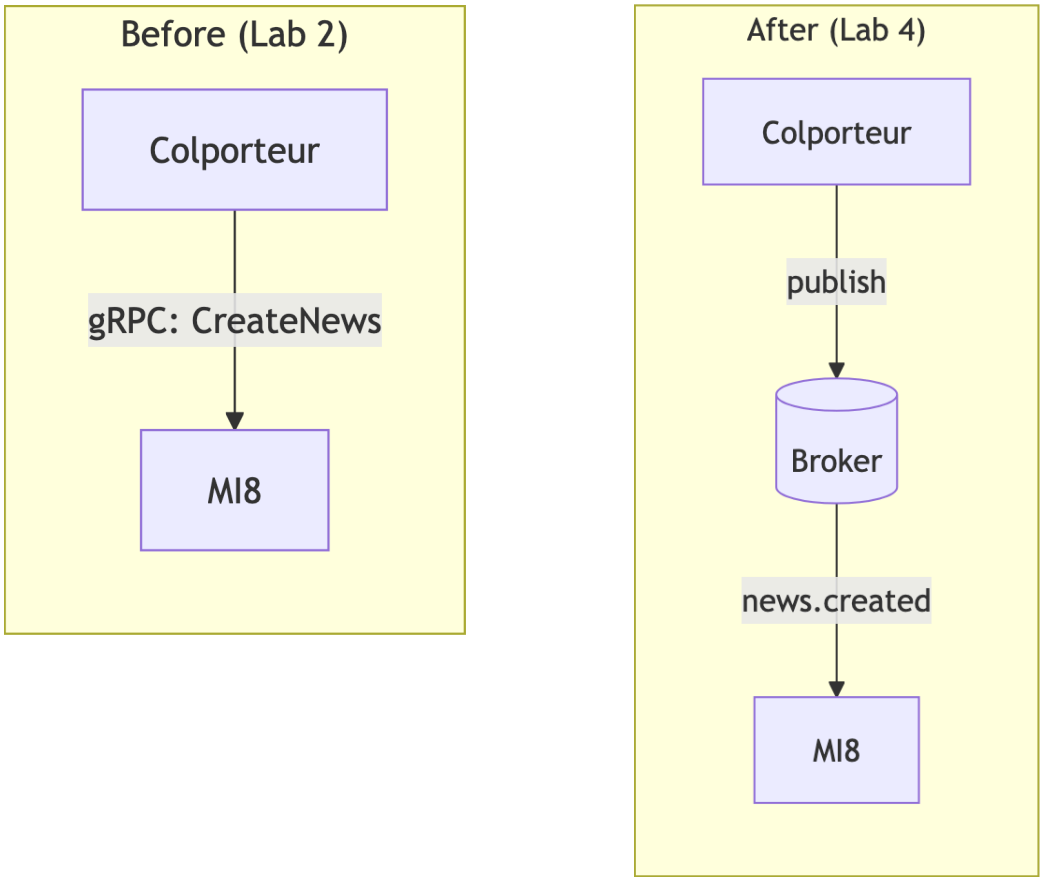
- Integrated a message broker (RabbitMQ) into the architecture
- Decoupled services through event-driven communication
- Created a new notification service that reacts to events

Part 1: News Created Flow

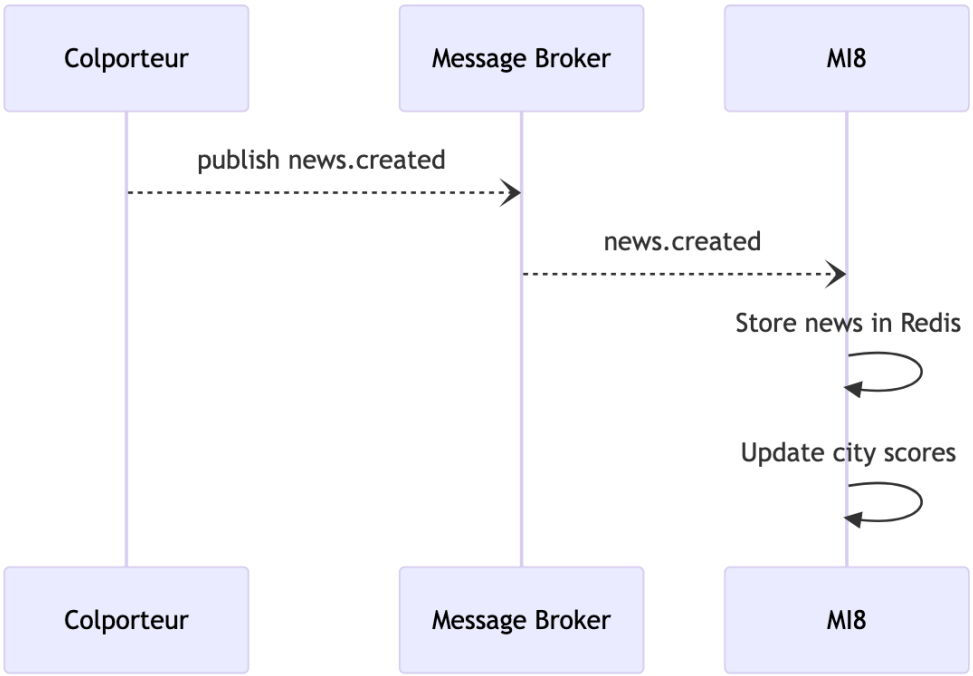
Objective

Transform the Colporteur → MI8 communication from gRPC to message-driven. Add RabbitMQ as a broker to your infrastructure.

- The broker must be accessible by all services
- Create the new topics needed to exchange about the news creation
- Modify or create the Colporteur script that will publish news into this topic
- MI8 should now subscribe to the topic for news creation



Flow Diagram



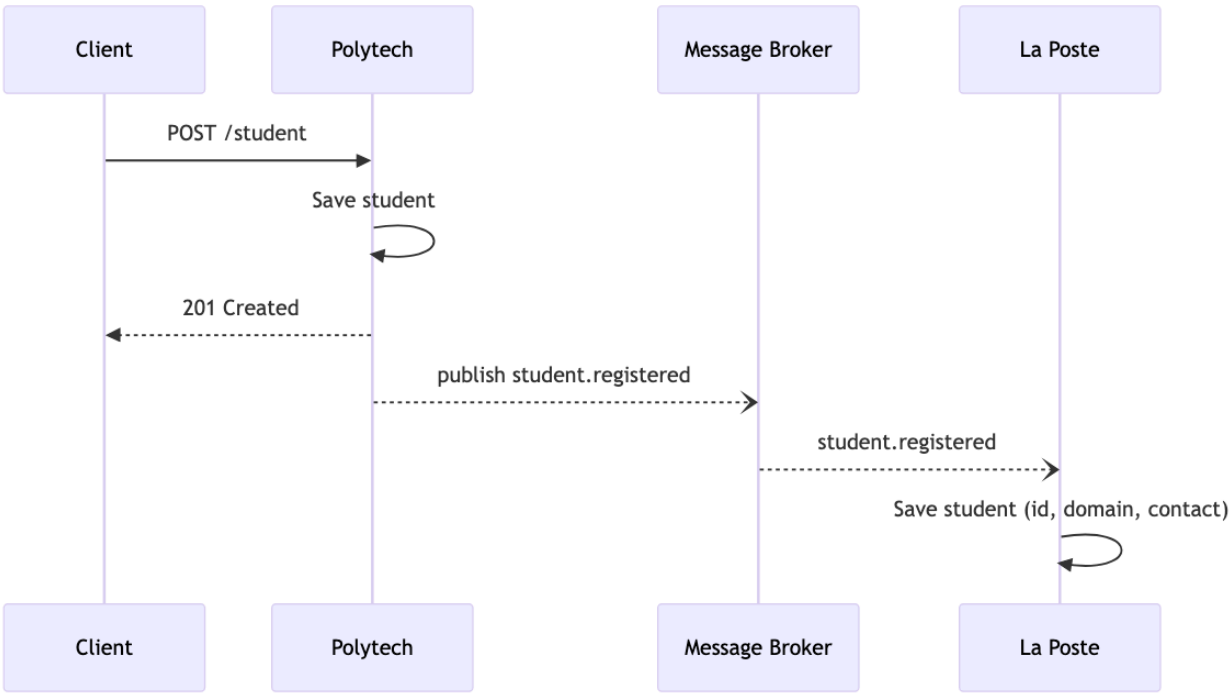
Part 2: Implement La Poste Service

Objective

Create the La Poste service that will send notifications to students. Before it can notify anyone, it needs to know who the students are.

When a student registers in Polytech, La Poste needs to be informed. Polytech will publish a `student.registered` event.

Implement the needed topics, publisher subscriber and the new La Poste service



Event Structure

Student.registered

- StudentId
- Name
- Domain
- createdAt

La Poste REST API

La Poste also exposes a REST API for students to manage their notification preferences.

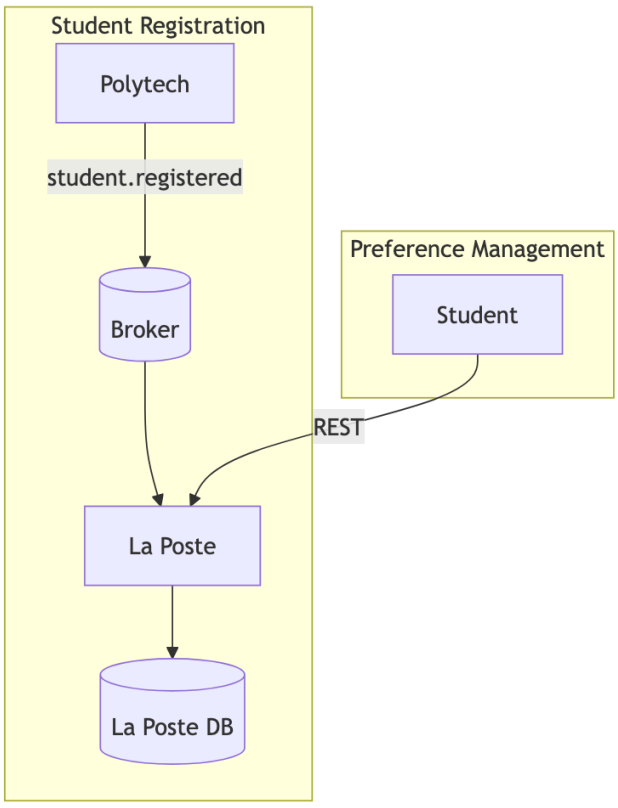
Data Model: Subscriber

- StudentId
- Domain
- Channel (discord/email)
- Contact: contact information
- Enabled (boolean): notification enabled

Endpoints

Method	Endpoint	Description
`GET`	`/subscribers/:studentId`	Get subscriber preferences
`PUT`	`/subscribers/:studentId`	Update notification preferences
`DELETE`	`/subscribers/:studentId`	Unsubscribe from notifications

Flow Diagram

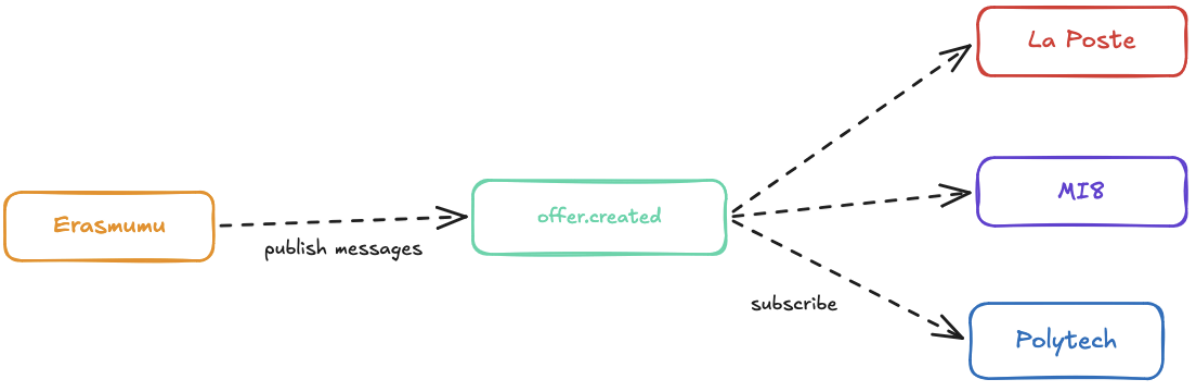


Part 3: Offer Created Flow

Objective

When a new internship offer is created in Erasmumu, multiple services react to this event independently.

Architecture Diagram



What Each Service Does

- **Polytech** Store notification per matching student
- **MI8**: Increment city offer counter
- **La Poste:** Send alert

Polytech: Student Notifications

Data Model: Notification

- Id
- studentId
- Type (new_offer)
- offerId
- Message: notification text
- Read (boolean)

New Endpoints

Method	Endpoint	Description
GET	/students/:id/notifications	Get student notifications
PUT	/notifications/:id/read	Mark notification as read

MI8: City Statistics

Data Model: CityStats

Field	Type	Description
city	String	City name
totalOffers	Integer	Total offers posted
offersByDomain	Map	Count per domain
lastOfferDate	Timestamp	Most recent offer

New gRPC Method

- GetCityStats(string): CityStatCs

Instructions

Implement the offers creation flow with the changes on each services.
- Polytech should contains the new notifications REST endpoint and store them
- MI8 should expose the city stats

Part 4: Frontend Changes

Objective

Update the frontend application to reflect the new features: student notifications and La Poste preferences.

Feature 1: Notification Center

Add a notification panel where students can see alerts about new offers matching their domain.

Requirements:

- Fetch current preferences from La Poste `GET /subscribers/:studentId`
- Update preferences via `PUT /subscribers/:studentId`
- Unsubscribe via `DELETE /subscribers/:studentId`

Feature 2: La Poste Preferences

Add a settings page where students can configure their notification preferences.

Part 5 (Bonus): Real-time Frontend Updates

Objective

Push live updates to the frontend when news is created.

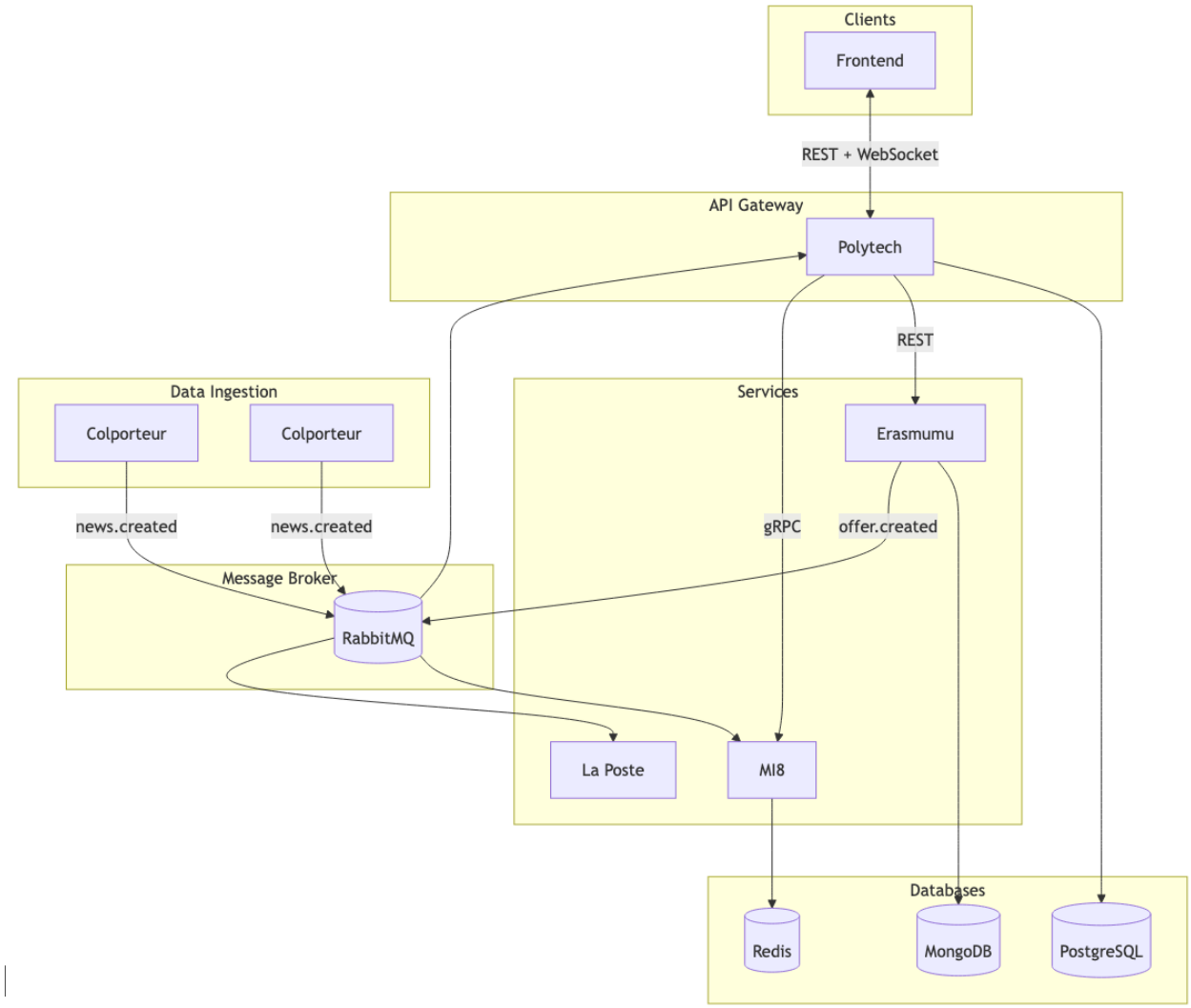
Requirements

- Polytech subscribes to `news.created`
- Polytech maintains WebSocket connections with frontends
- When news arrives, push to connected clients
- Frontend displays a toast/notification with the news headline

Deliverable

- Live news notifications appear in the frontend without refreshing

Final Architecture



Evaluation Criteria

Broker Setup

- Message broker is running and accessible
- Services can connect and publish/subscribe

Part 2: News Created Flow

- Colporteurs publish events instead of gRPC
- MI8 processes news from events
- Scores still update correctly

Part 3: La Poste Service

- Polytech publishes `student.registered` on registration
- La Poste subscribes and stores students
- REST API for notification preferences works

Part 4: Offer Created Flow

- Erasmumu publishes event on offer creation
- Polytech creates notifications for matching students
- MI8 updates city statistics
- La Poste sends mock alerts to subscribed students
- All consumers are independent (one failing doesn't affect others)

Part 5: Frontend Changes

- Notification center displays student alerts
- Notifications can be marked as read
- La Poste preferences page works
- Students can update channel and contact info
-

Deliverable

Once completed, commit your work to a branch named:
t`lab4-messaging`

Be sure to put your repository in public or invite me with my GridexX